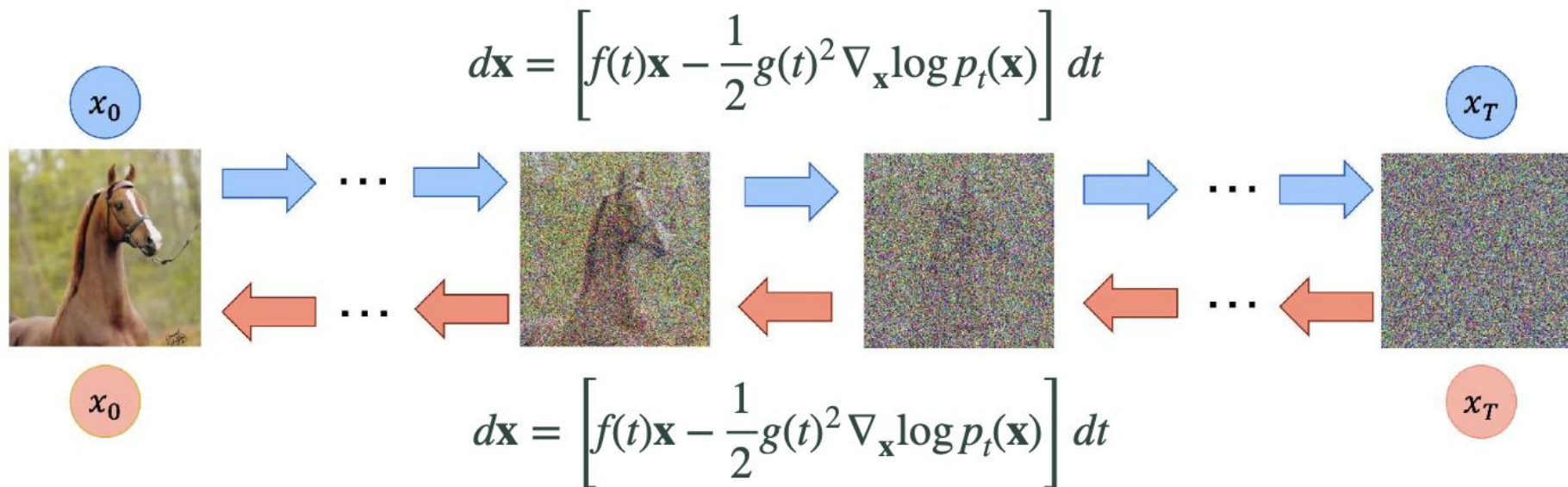


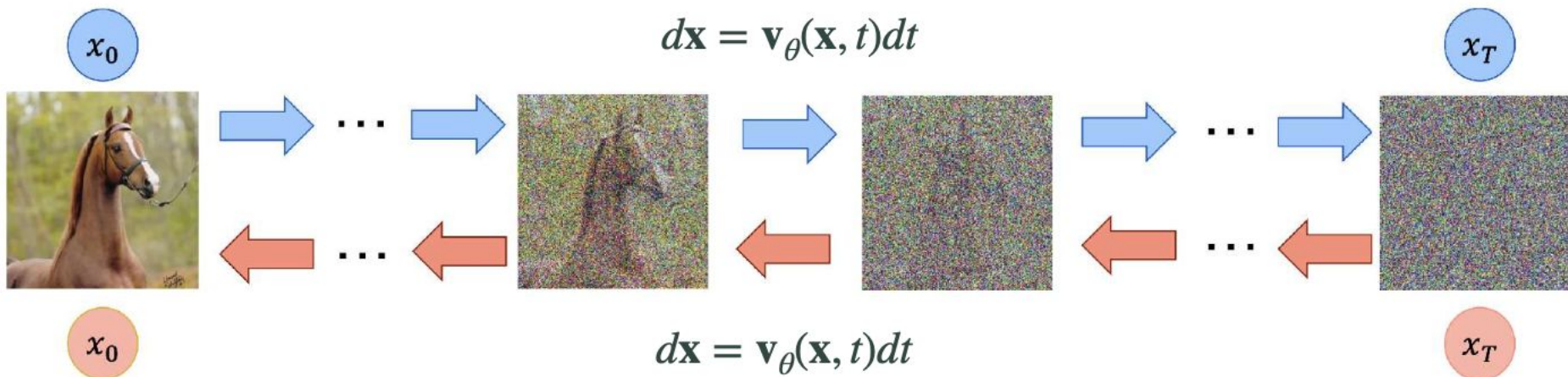
ODE-free
few-step models

Recap



Recap

Flow Matching. Numerical solvers offer 15-20 step sampling. What about 1-4 steps?

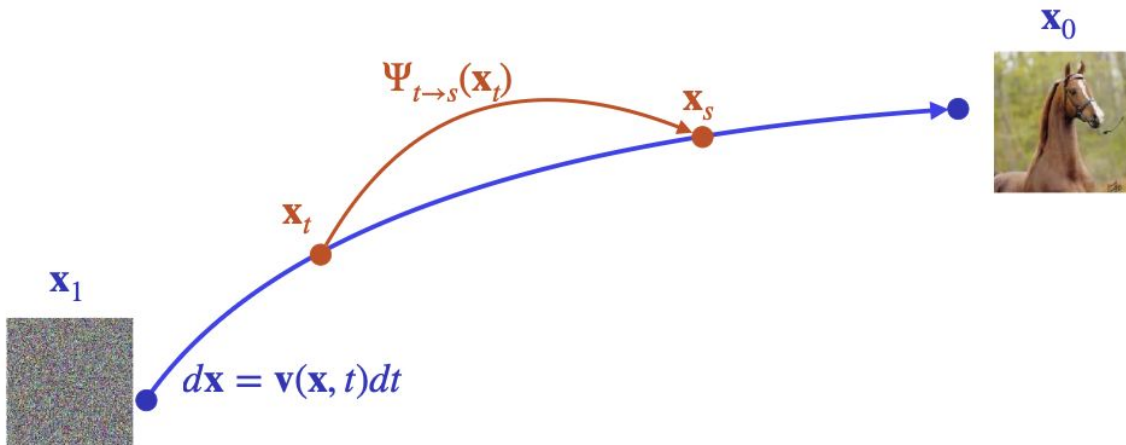


Recap

Flow Map: $\Psi_{t \rightarrow s}(\mathbf{x}_t) = \mathbf{x}_t + \int_t^s \mathbf{v}(\mathbf{x}, u) du$, oracle integrator for $d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)dt$

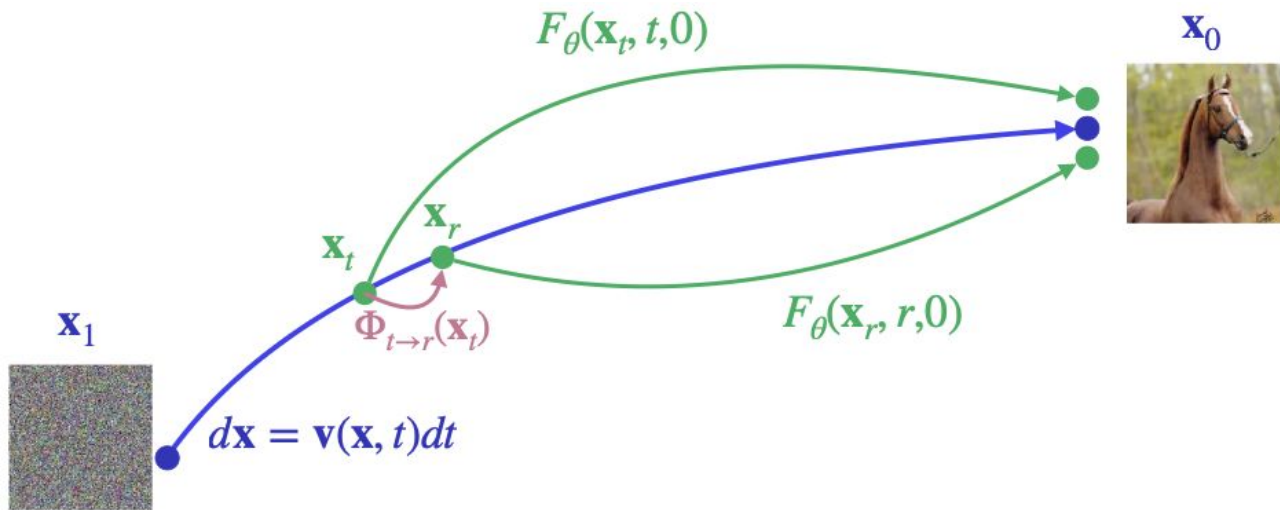
Goal: train $F_\theta(\mathbf{x}_t, t, s) \approx \Psi_{t \rightarrow s}(\mathbf{x}_t)$

How? Design via loss integral properties



Recap

Consistency models: $\|F_\theta(\mathbf{x}_t, t, 0) - \text{sg}(F_\theta(\mathbf{x}_r, r, 0))\|_2^2$



Notation

Generator $G_{\theta}(\mathbf{z})$ produces $p_{fake}(\mathbf{x})$, where $\mathbf{z} \sim \mathcal{N}(0, I)$

Data distribution $p_{real}(\mathbf{x})$

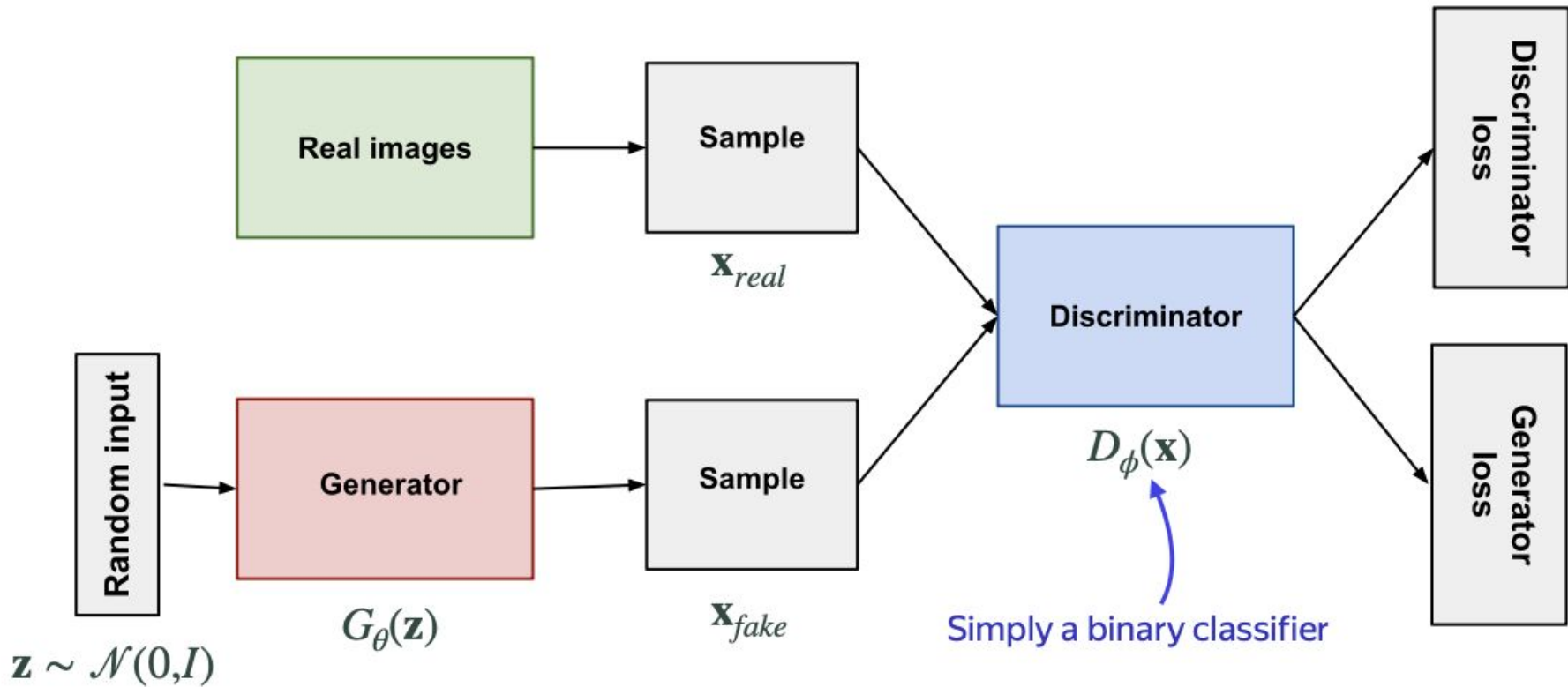
Forward process: $q(\mathbf{x}_t | \mathbf{x}) = \mathcal{N}(\alpha_t \mathbf{x}, \sigma_t^2 I)$

Pretrained diffusion model (DM): $\epsilon_{\psi}(\mathbf{x}_t, t)$

Tweedie's formula:

$$\nabla_{\mathbf{x}_t} \log p_{real}(\mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) = \mathbb{E} \left[\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}) | \mathbf{x}_t \right] \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}) = - \frac{\mathbf{x}_t - \alpha_t \mathbf{x}}{\sigma_t^2} = - \frac{\epsilon}{\sigma_t}$$

Recap of GANs



Recap of GANs

Standard Min-Max loss

Discriminator loss:

$$\max_{\phi} \left[\mathbb{E}_{\mathbf{x} \sim p_{\text{real}}} \log D_{\phi}(\mathbf{x}) + \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, I)} \log(1 - D_{\phi}(G_{\theta}(\mathbf{z}))) \right]$$

Generator loss:

$$\min_{\theta} \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, I)} \log(1 - D_{\phi}(G_{\theta}(\mathbf{z})))$$

Non-saturating generator loss:

$$\min_{\theta} \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, I)} -\log D_{\phi}(G_{\theta}(\mathbf{z})))$$

Recap of GANs

GANs are powerful **single-step** generative models

but **struggle to achieve SOTA performance** without some help, e.g. **diffusion**

Distribution Matching Distillation.

Training objective

Let's consider **reverse KL objective**

$$D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{p_{fake}(\mathbf{x})} [\log p_{fake}(\mathbf{x}) - \log p_{real}(\mathbf{x})] = \underbrace{\mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, I)}}_{\text{Reparameterization trick}} [\log p_{fake}(\underline{G_{\theta}(\mathbf{z})}) - \log p_{real}(\underline{G_{\theta}(\mathbf{z})})]$$

$\log p_{fake}(\mathbf{x})$ and $\log p_{real}(\mathbf{x})$ are unavailable

Distribution Matching Distillation.

Training objective

$$D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\log p_{fake}(G_{\theta}(\mathbf{z})) - \log p_{real}(G_{\theta}(\mathbf{z})) \right]$$

$$\nabla_{\theta} D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\nabla_{\theta} \log p_{fake}(G_{\theta}(\mathbf{z})) - \nabla_{\theta} \log p_{real}(G_{\theta}(\mathbf{z})) \right] = \text{Chain rule}$$

$$= \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\underbrace{(\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x}))}_{\text{blue}} - \underbrace{\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})}_{\text{blue}} \right] \nabla_{\theta} G_{\theta}(\mathbf{z}), \quad \text{where } \mathbf{x} = G_{\theta}(\mathbf{z})$$

Can we compute $\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$ and $\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})$?

Distribution Matching Distillation.

Training objective

$$D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\log p_{fake}(G_{\theta}(\mathbf{z})) - \log p_{real}(G_{\theta}(\mathbf{z})) \right]$$

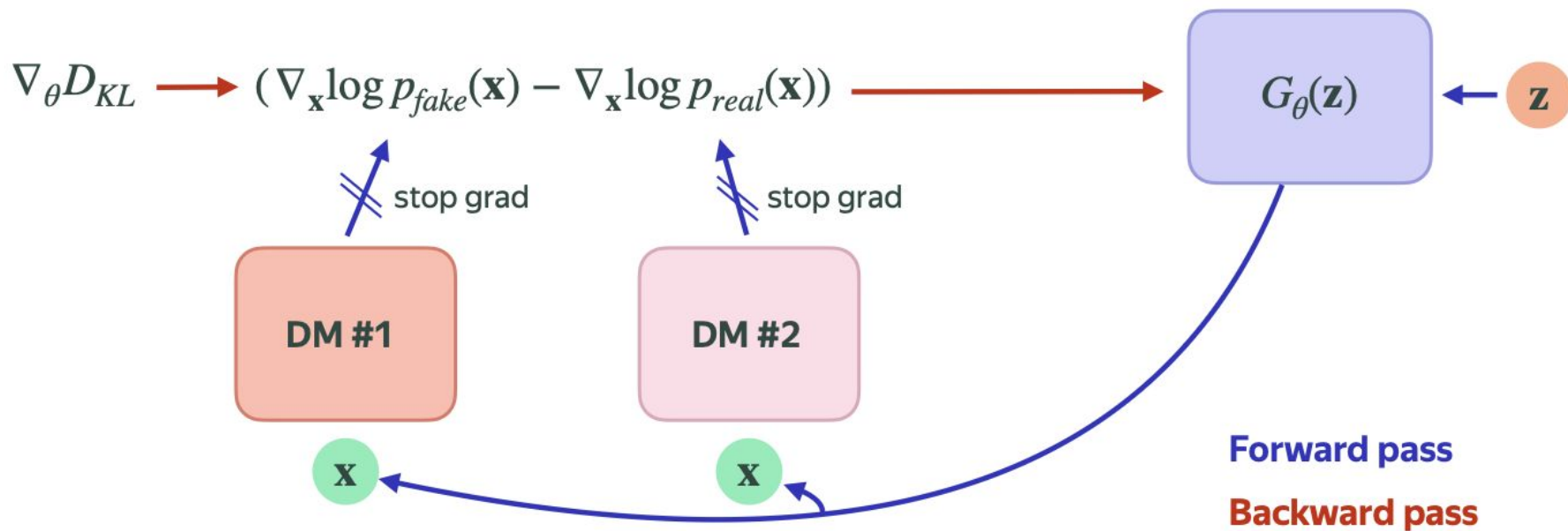
$$\nabla_{\theta} D_{KL}(p_{fake} \| p_{real}) = \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\nabla_{\theta} \log p_{fake}(G_{\theta}(\mathbf{z})) - \nabla_{\theta} \log p_{real}(G_{\theta}(\mathbf{z})) \right] = \text{Chain rule}$$

$$= \mathbb{E}_{z \sim \mathcal{N}(0, I)} \left[\left(\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x}) - \nabla_{\mathbf{x}} \log p_{real}(\mathbf{x}) \right) \nabla_{\theta} G_{\theta}(\mathbf{z}) \right], \quad \text{where } \mathbf{x} = G_{\theta}(\mathbf{z})$$

Use DM forward passes to estimate $\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$ and $\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})$

Distribution Matching Distillation.

Generator update



Distribution Matching Distillation.

Generator update

Important: DM is pretrained on $p_{real}(\mathbf{x})$, so it can estimate only $\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})$

How to estimate $\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$?

Key idea

Let's train a **new** DM $s_\phi(\mathbf{x}_t, t)$ on $p_{fake}(\mathbf{x})$, produced by **current** G_θ

Key idea

Let's train a **new** DM $s_\phi(\mathbf{x}_t, t)$ on $p_{fake}(\mathbf{x})$, produced by **current** G_θ

Problem: $p_{fake}(\mathbf{x})$ changes after each G_θ update

Key idea

Let's train a **new** DM $s_\phi(\mathbf{x}_t, t)$ on $p_{fake}(\mathbf{x})$, produced by **current** G_θ

Problem: $p_{fake}(\mathbf{x})$ changes after each G_θ update

Solution: re-train $s_{fake}^\phi(\mathbf{x}_t, t)$ after each G_θ update

For efficiency, a single update per training iteration is okay

Algorithm

1. Update step for G_θ

$$\nabla_\theta D_{KL} \simeq \mathbb{E}_{\mathbf{x} \sim G_\theta(\mathbf{z})} \left[(s_{fake}^\phi(\mathbf{x}, 0) - s_{real}^\psi(\mathbf{x}, 0)) \nabla_\theta G_\theta(\mathbf{z}) \right]$$

2. Update step for fake DM ϵ_ϕ

Typical DM training step

$$\nabla_\phi \mathbb{E}_{\mathbf{z}, t, \mathbf{x}, \mathbf{x}_t} \|s_{fake}^\phi(q(\mathbf{x}_t | \mathbf{x}), t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x})\|_2^2 = \nabla_\phi \mathbb{E}_{\mathbf{z}, t, \mathbf{x}, \mathbf{x}_t} \|\epsilon_\phi(q(\mathbf{x}_t | \mathbf{x}), t) - \epsilon\|_2^2$$

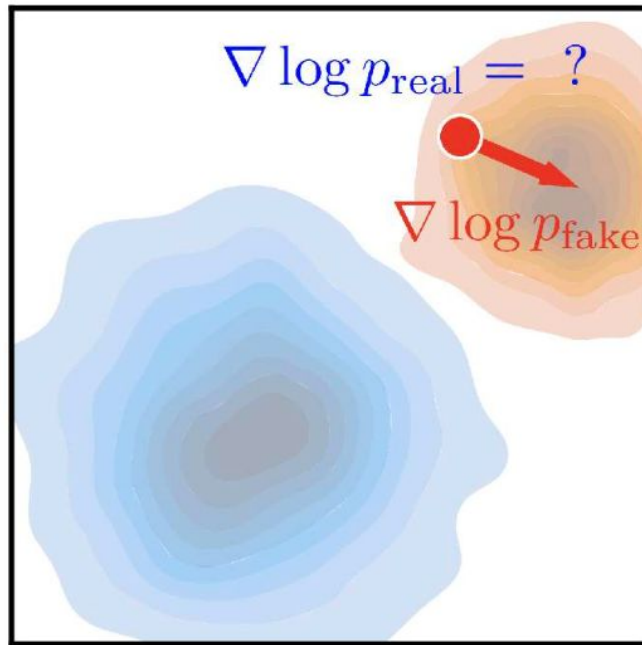
What is a problem here?

Undefined Scores

G_θ produces low quality samples at the beginning

Pretrained DM observed nearly real samples for $t = 0$

$\rightarrow s_{real}^\psi(\mathbf{x}, 0)$ is inaccurate for $\mathbf{x} \sim p_{fake}(\mathbf{x})$

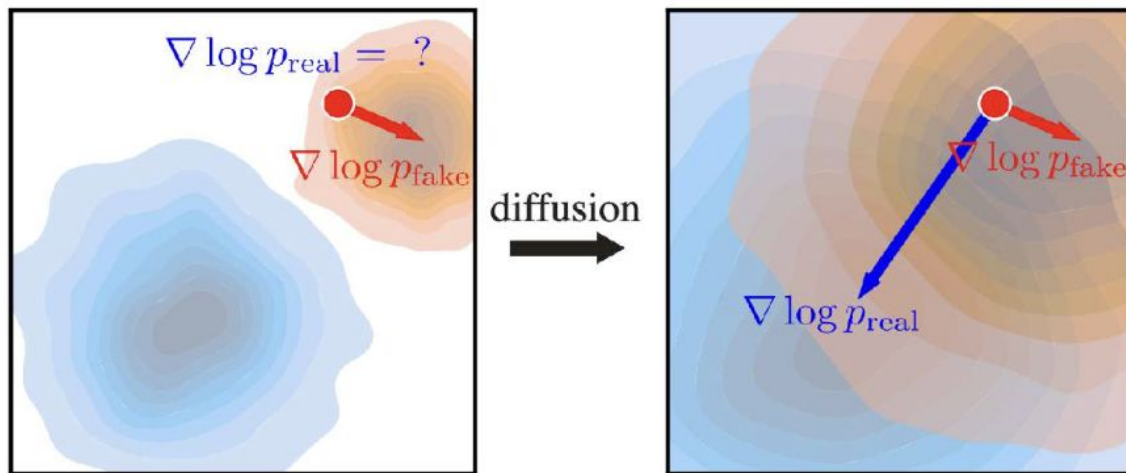


Undefined Scores

Similar idea to the annealed sampling in denoising score matching

Let's use $t \gg 0$ to avoid undefined scores at the beginning, e.g., $t = T$

Gradually reduce t once G_θ gets better



Overall Algorithm

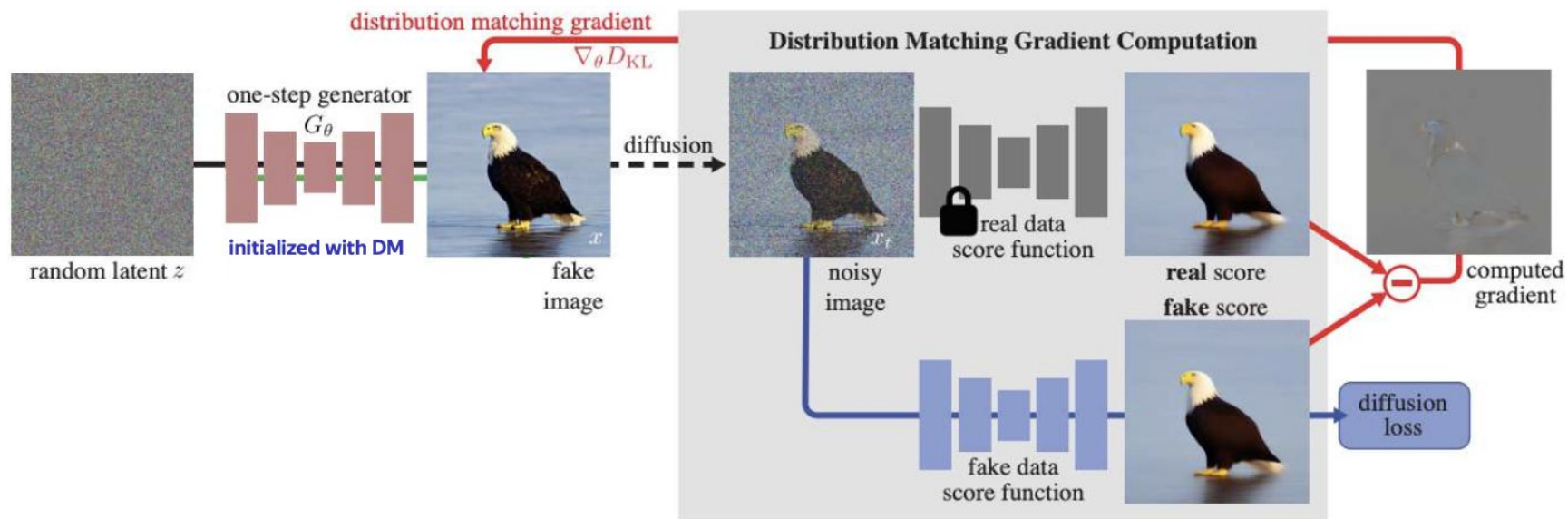
1. Update step for G_θ

$$\nabla_\theta D_{KL} \simeq \mathbb{E}_{\mathbf{z}, t, \mathbf{x}, \mathbf{x}_t} \left[(s_{fake}^\phi(\mathbf{x}_t, t) - s_{real}^\psi(\mathbf{x}_t, t)) \nabla_\theta G_\theta(\mathbf{z}) \right]$$

2. Update step for fake DM ϵ_ϕ

$$\nabla_\phi \mathbb{E}_{\mathbf{x} \sim G_\theta(\mathbf{z})} \|s_{fake}^\phi(q(\mathbf{x}_t | \mathbf{x}), t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x})\|_2^2 = \nabla_\phi \mathbb{E}_{\mathbf{x} \sim G_\theta(\mathbf{z})} \|\epsilon_\phi(q(\mathbf{x}_t | \mathbf{x}), t) - \epsilon\|_2^2$$

Overall Algorithm



DMD loss

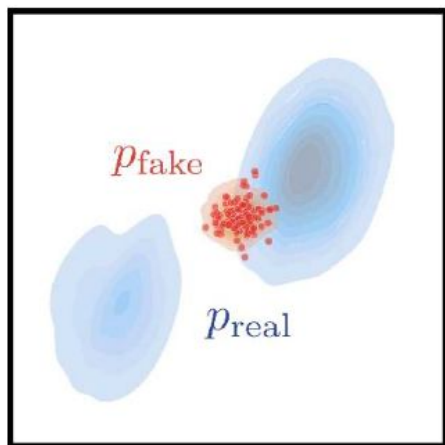
Practical detail: how to implement $\nabla_{\theta} D_{KL}$?

Surrogate loss:
$$L = \frac{1}{2} \mathbb{E}_{\mathbf{z}, t, \mathbf{x}, \mathbf{x}_t} \left[\|G_{\theta}(\mathbf{z}) - \text{sg} \left(G_{\theta}(\mathbf{z}) - s_{fake}^{\phi}(\mathbf{x}_t, t) + s_{real}^{\psi}(\mathbf{x}_t, t) \right)\|_2^2 \right]$$

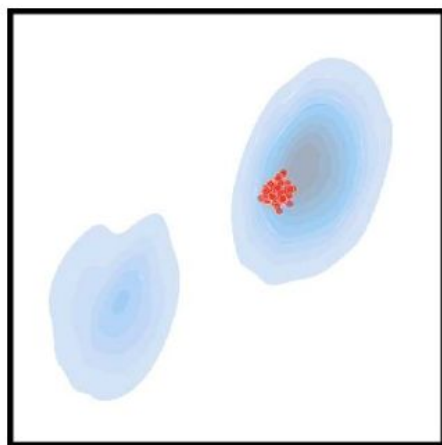
It can be shown that $\nabla_{\theta} L = \nabla_{\theta} D_{KL}$ via chain rule.

Reverse KL problem

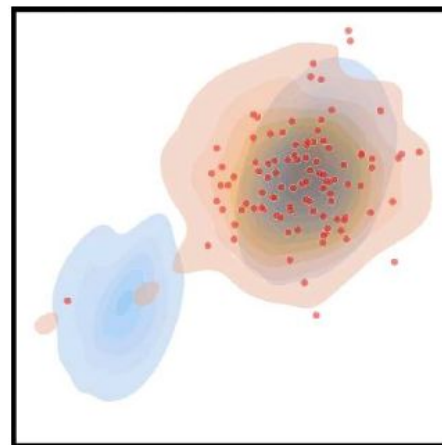
$$\mathbb{E}_{\substack{z \sim \mathcal{N}(0; \mathbf{I}) \\ x = G_{\theta}(z)}} - (\log p_{\text{real}}(x) - \log p_{\text{fake}}(x))$$



Initial state



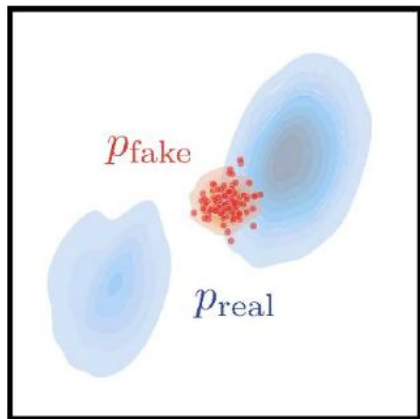
Real score only



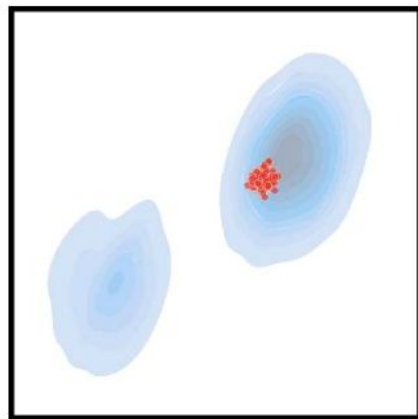
Real + fake scores

Reverse KL problem

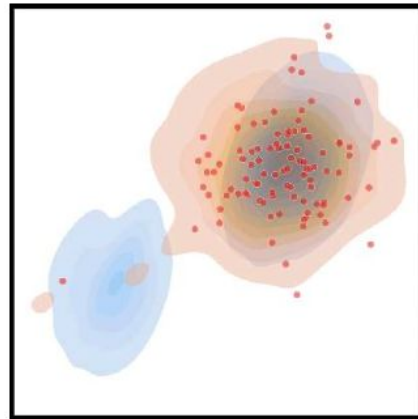
How to avoid mode collapse?



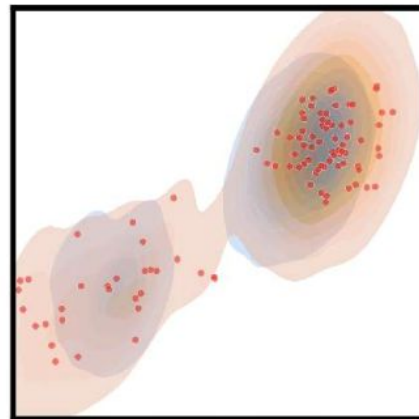
Initial state



Real score only



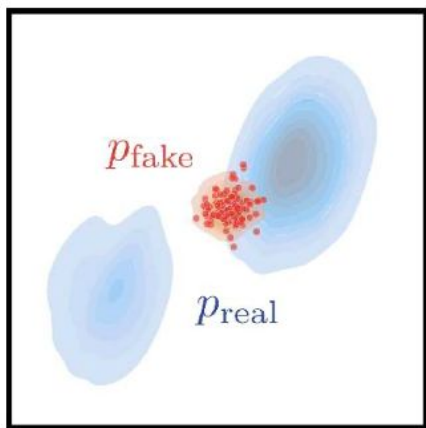
Real + fake scores



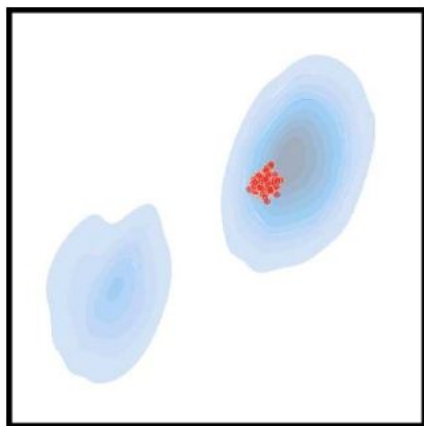
?

Reverse KL problem

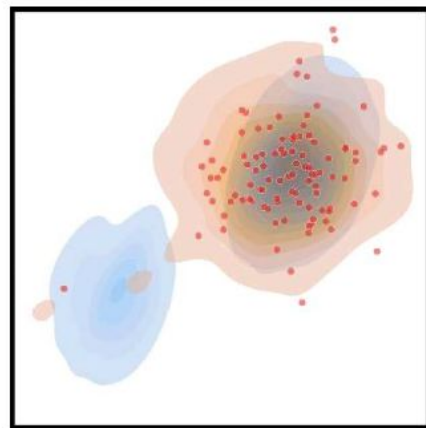
Additionally use a classical Knowledge Distillation approach



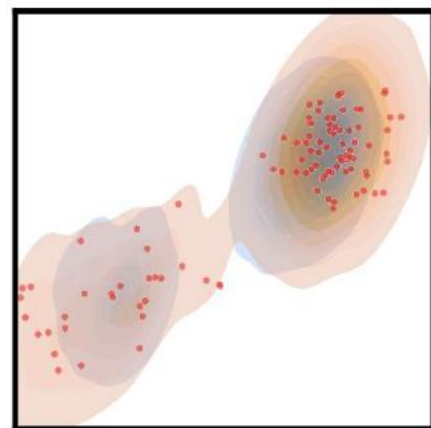
Initial state



Real score only

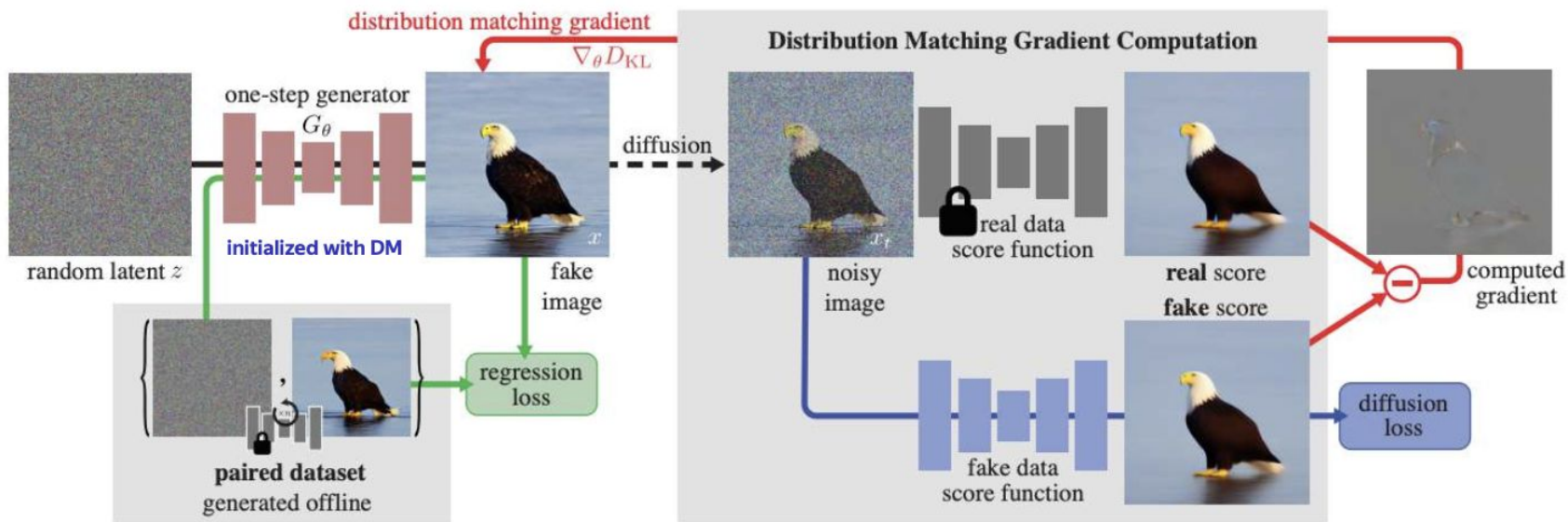


Real + fake scores



Real + fake scores + KD

Distribution Matching Distillation



Score Distillation Sampling (SDS)

Can we get rid of the fake DM?

Let's use unbiased score function estimation for $s_{fake}(\mathbf{x}_t, t)$ on $\mathbf{x} \sim G_\theta(\mathbf{z})$:

$$s_{fake}(\mathbf{x}_t, t) = \nabla_{\mathbf{x}_t} \log p_{fake}(\mathbf{x}_t) = \mathbb{E}_{\mathbf{x} \sim p_{fake}} [\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}) | \mathbf{x}_t] \approx \nabla_{x_t} \log q(\mathbf{x}_t | \mathbf{x}) = - \frac{\mathbf{x}_t - \alpha_t \mathbf{x}}{\sigma_t^2}$$

- **Performs poorly alone** → used only as a minor polishing loss
- Alternative cheap ways to avoid the fake DM are still unsuccessful

Score Distillation Sampling (SDS)

$$\mathcal{L}_{SDS} = \mathbb{E}_{t,\epsilon} \left[w(t) ||s_{\theta}(\mathbf{x}_t, t) - s_t(\mathbf{x}_t, t)||^2 \right]$$

DMD summary

Key idea: use DM forward passes to estimate $\nabla_{\mathbf{x}} \log p_{fake}(\mathbf{x})$ and $\nabla_{\mathbf{x}} \log p_{real}(\mathbf{x})$

Pros:

- Fruitful idea for future research and promising results

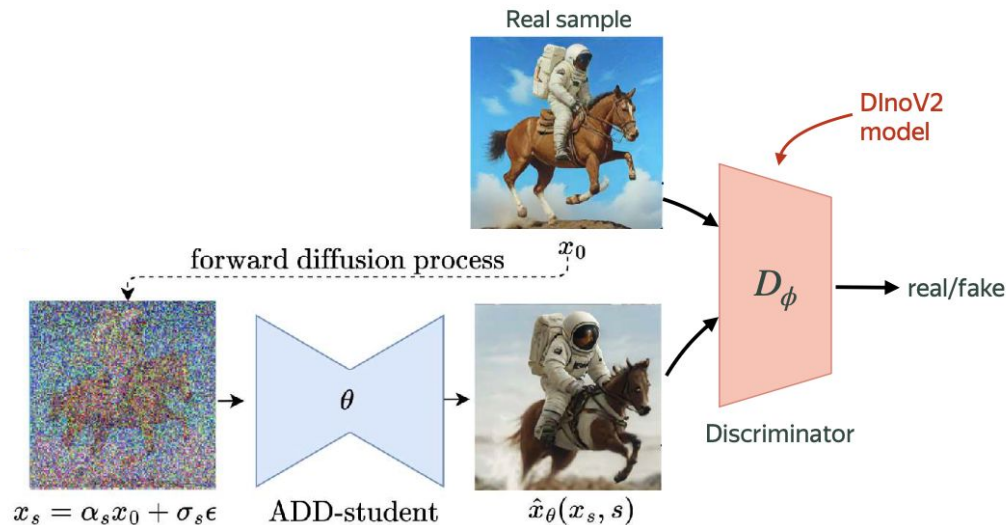
Cons:

- Requires additional DM training in parallel
- Relatively expensive and unstable
- Prone to mode collapse → needs additional losses

Adversarial Diffusion Distillation

1. $G_{\theta}(\mathbf{z})$ is initialized with pretrained Diffusion Model
2. Multistep sampling via forward process $s \in T_{student} = \{\tau_1, \dots, \tau_n\}$

Adversarial Diffusion Distillation = GAN for $n=1$ and $\text{tau}=1000$

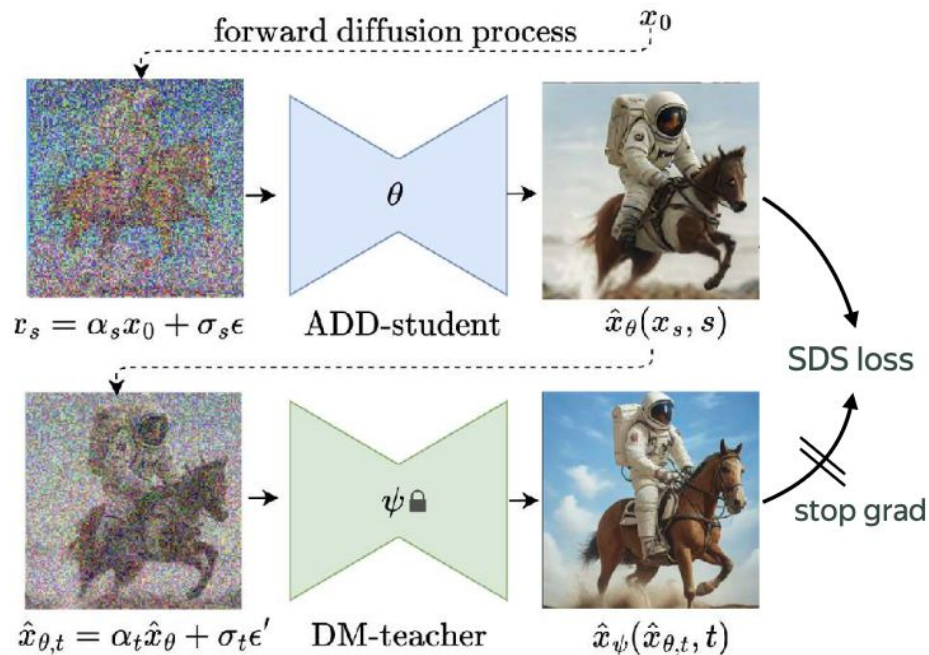


Adversarial Diffusion Distillation

Combine GAN and SDS losses

Here SDS is reparametrized via x_0 -pred

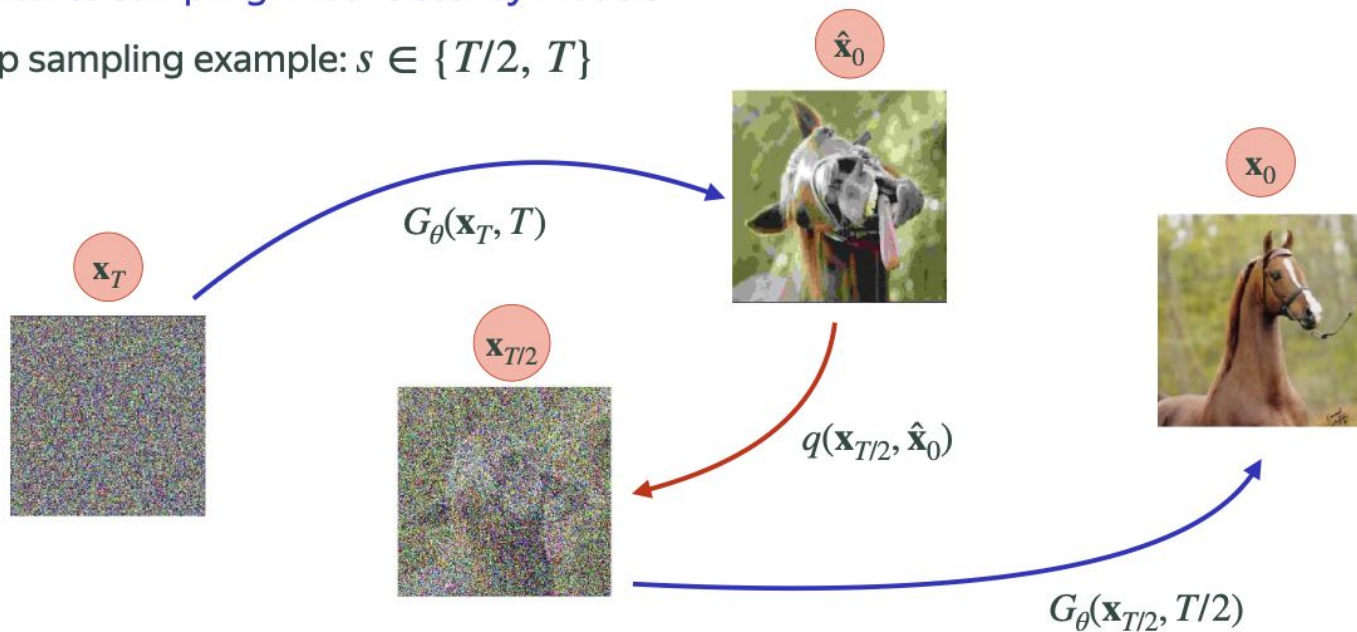
SDS plays a minor role in the final perf.



ADD sampling

Identical to sampling in Consistency Models

2-step sampling example: $s \in \{T/2, T\}$



ADD summary

Example: SDXL-Turbo

Key idea: DM-initialized GAN + SDS for negligible polishing

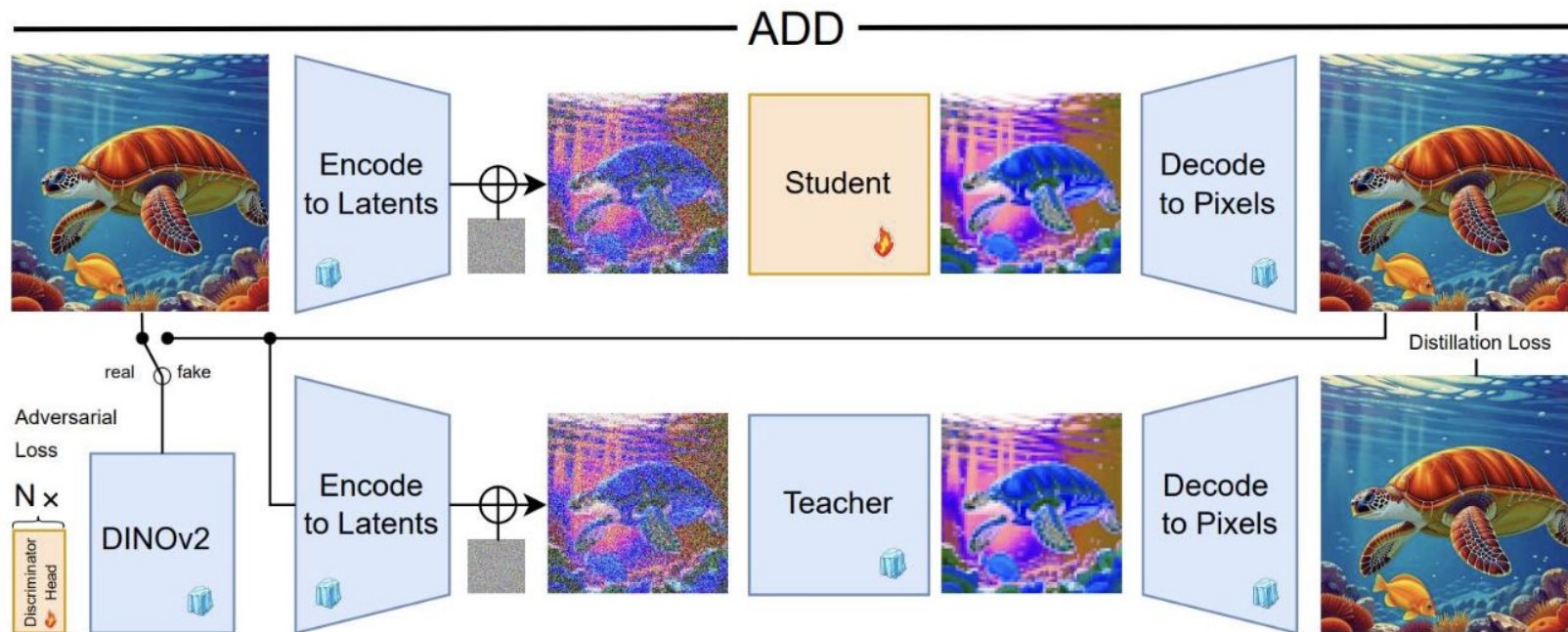
Both objectives are prone to mode collapse → poor sample diversity for a given prompt

Woman bent slightly on skis wearing goggles and snowsuit.

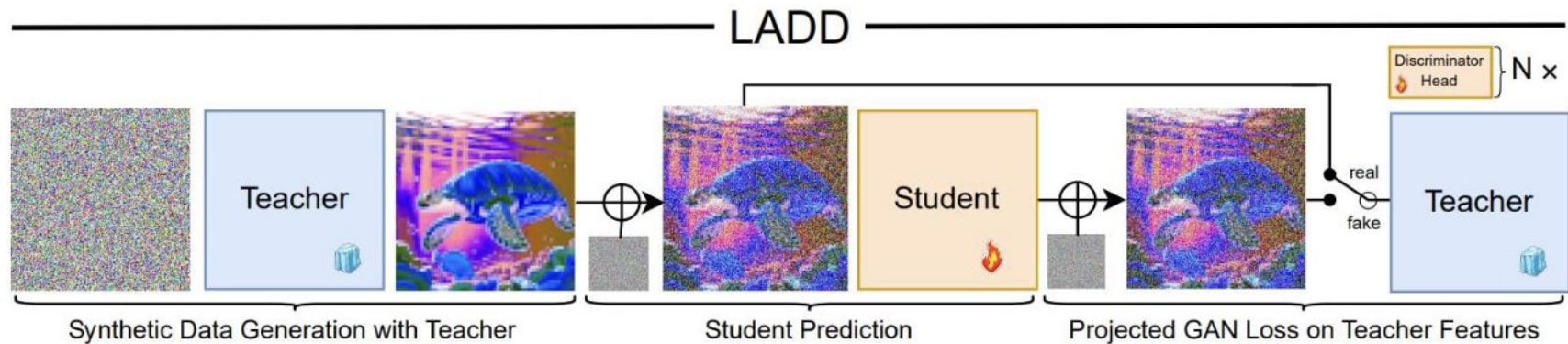
ADD-XL



ADD summary



Latent Adversarial Diffusion Distillation



Latent Adversarial Diffusion Distillation

LADD introduces 2 main modifications:

1. unification of discriminator and teacher model
2. adoption of synthetic data for training

LADD vs ADD

Stays in VAE latent space → more efficient training

Discriminator is initialized with a teacher DM → feedback from different noise levels

SDS is not needed

LADD is a reasonable choice in production

What about ...



DMD

LADD

Improved DMD (DMD2)

Let's combine GAN and DMD losses

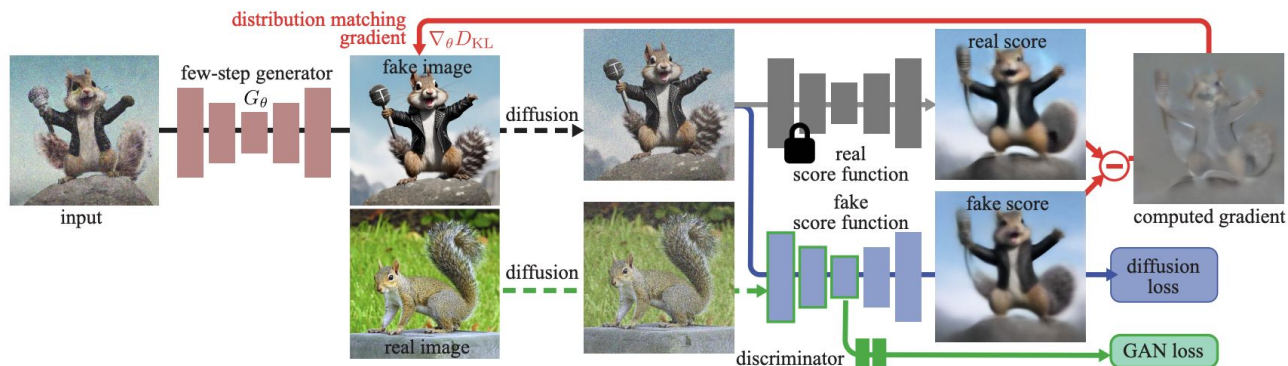


Figure 3: Our method distills a costly diffusion model (gray, right) into a one- or multi-step generator (red, left). Our training alternates between 2 steps: 1. optimizing the generator using the gradient of an implicit distribution matching objective (red arrow) and a GAN loss (green), and 2. training a score function (blue) to model the distribution of “fake” samples produced by the generator, as well as a GAN discriminator (green) to discriminate between fake samples and real images. The student generator can be a one-step or a multi-step model, as shown here, with an intermediate step input.

Improved DMD (DMD2)

Let's combine GAN and DMD losses

We address this issue by incorporating an additional GAN objective into our pipeline, where the discriminator is trained to distinguish between *real* images and images produced by our generator. Trained using real data, the GAN classifier does not suffer from the teacher's limitation, potentially allowing our student generator to surpass it in sample quality. Our integration of a GAN classifier into DMD follows a minimalist design: we add a classification branch on top of the bottleneck of the

fake diffusion denoiser (see Fig. 3). The classification branch and upstream encoder features in the UNet are trained by maximizing the standard non-saturating GAN objective:

$$\mathcal{L}_{\text{GAN}} = \mathbb{E}_{x \sim p_{\text{real}}, t \sim [0, T]} [\log D(F(x, t))] + \mathbb{E}_{z \sim p_{\text{noise}}, t \sim [0, T]} [-\log(D(F(G_{\theta}(z), t)))], \quad (4)$$

Improved DMD (DMD2)

DMD

- KD for stability and mode coverage
- Single step generator
- 1 update step for fake DM

DMD2

- GAN
- Multistep generator
- 5 update steps for fake DM (important)

DMD2 results

4-step DMD2-Qwen-Image



4-step DMD2-SDXL



Takeaways

DMD — Use DMs for reverse KL gradient estimation

ADD, LADD — Diffusion-based GANs, where DM initialization plays a crucial role

DMD2 — DMD + GAN loss — current state-of-the-art

All of them trade diversity for image fidelity but that is what we are okay to trade in practice

All of them can outperform their teachers due to lower coverage

